Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

ЗВІТ

з лабораторної роботи №6
з дисципліни «Сучасні мобільні операційні системи»

Тема: «Повідомлення»

Варіант №4

Виконав:
студент 1 курсу, групи ІМ-51мн
Ковальов Олександр

Перевірив:
асистент, Нестерук Андрій Олександрович

Дата здачі: 22.03.2026

КИЇВ – 2026

Мета роботи. Вивчити інструменти зберігання даних та реалізації повідомлень.

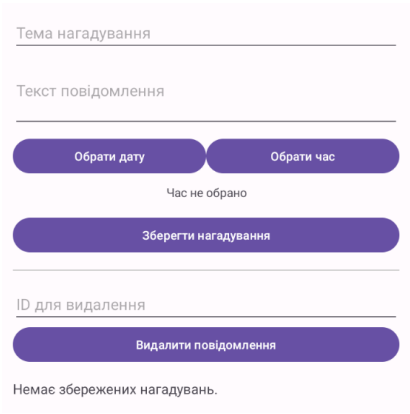
Завдання. Розробити мобільну прикладну програму, що дозволяє встановити нагадування.

Вимоги до програми:

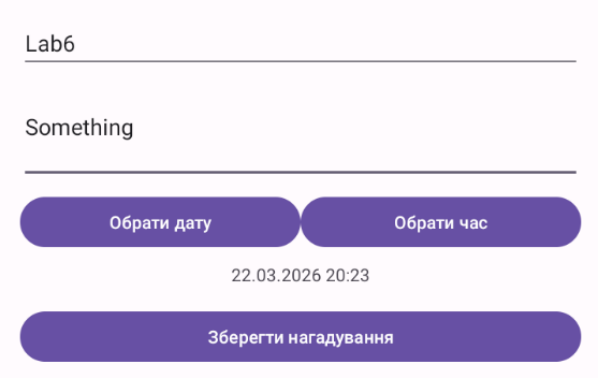
1. Створення нагадувань і збереження їх в базу даних. В базі даних повинні зберігатися наступні значення (`Тема`, `Текст повідомлення`, `Дата повідомлення`);
2. Перегляд повідомлень;
3. Видалення повідомлень;
4. Дата нагадування повинна встановлюватися за допомогою `TimePickerDialog` і `DatePickerDialog`;
5. Стилізувати нагадування в Notification Center і Status bar (встановити власне лого);
6. При натисканні на повідомлення в Notification Center переходити в активацію додатку з повним текстом повідомлення.

Хід роботи.

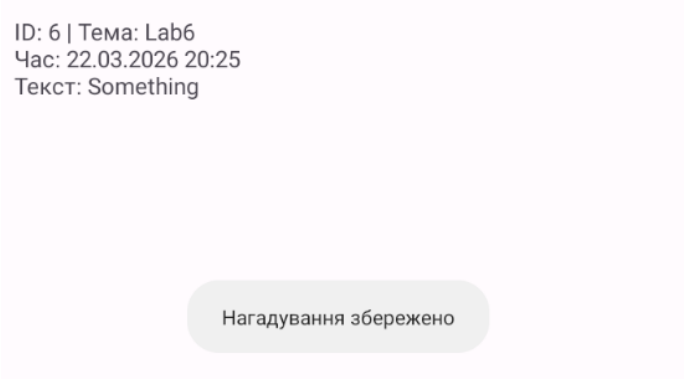
Розробка програми розпочалася з налаштування системного маніфесту, куди було додано необхідні дозволи для відправки сповіщень та використання точних системних таймерів, а також зареєстровано додаткову активність і приймач широкомовних повідомлень. Для персистентного збереження нагадувань було розроблено клас помічника бази даних SQLite, у якому створено таблицю з полями для ідентифікатора, теми, тексту та часу події. Цей клас інкапсулює логіку додавання нових записів, форматованого зчитування всієї історії для виведення на екран та видалення повідомлень за вказаним ідентифікатором. Графічний інтерфейс головної активності забезпечує взаємодію з користувачем через текстові поля та кнопки керування.



Для зручного та безпомилкового вибору дати і часу спрацьовування було інтегровано системні діалоги `DatePickerDialog` та `TimePickerDialog`, результати яких зберігаються в об’єкті класу `Calendar`.



При збереженні нового нагадування програма не лише робить запис у базу даних, але й конфігурує системний сервіс `AlarmManager`, встановлюючи точний час пробудження додатку.



У визначений момент цей сервіс ініціює спрацьовування класу `ReminderReceiver`, який успадковує `BroadcastReceiver`. Всередині цього приймача створюється канал сповіщень (обов’язкова вимога для сучасних версій Android) та формується саме повідомлення за допомогою `NotificationCompat.Builder`, куди передається спеціально підготовлена власна іконка для відображення у Status bar та Notification Center.

Для реалізації переходу до повного тексту нагадування до сповіщення прив'язується об'єкт `PendingIntent`. При натисканні на повідомлення цей намір автоматично відкриває `DetailActivity`, передаючи їй тему та текст через екстрадані, які потім розпаковуються та відображаються у відповідних текстових полях на окремому екрані.

Висновок. Під час виконання лабораторної роботи було успішно розроблено мобільний додаток для створення та управління персональними нагадуваннями, що дозволило поглибити знання щодо фонової роботи операційної системи Android. На практиці закріплено навички використання локальної бази даних SQLite для збереження інформації про події. Значну увагу було приділено розробці користувацького інтерфейсу з інтеграцією системних діалогів `DatePickerDialog` та `TimePickerDialog` для безпомилкового вибору дати і часу. Головним етапом стало налаштування механізму відкладеного виконання завдань за допомогою системного сервісу `AlarmManager` та перехоплення його сигналів через `BroadcastReceiver`. Також було вивчено процес генерації та стилізації системних сповіщень за допомогою класу `NotificationCompat.Builder`, інтеграції власних графічних ресурсів та реалізації навігації у конкретне вікно додатку через механізм `PendingIntent` при натисканні на сповіщення користувачем.

Контрольні запитання.

1. **Як створити View-елемент з вмісту Layout-файлу? У яких випадках це необхідно?**

   Створення View-елемента з XML-файлу розмітки в оперативній пам'яті називається інфляцією і виконується за допомогою системного класу `LayoutInflater` та його методу `inflate`. Цей процес є критично необхідним у ситуаціях, коли інтерфейс генерується динамічно під час роботи програми, наприклад, при створенні кастомних рядків для списків `ListView` або `RecyclerView`, формуванні вмісту для спливаючих вікон `PopupWindow` та нестандартних діалогів, а також при роботі з фрагментами, де візуальна частина не може бути автоматично прив'язана через метод `setContentView`, як це зазвичай відбувається у звичайних активностях.

2. **Як створити і забезпечити роботу списку `ListView`?**

   Для забезпечення роботи списку `ListView` необхідно спочатку додати відповідний тег у файл XML-розмітки екрана. Далі у коді активності потрібно знайти цей елемент за його ідентифікатором та підготувати джерело даних,

яким може бути масив або список об'єктів. Зв'язування даних із візуальним компонентом здійснюється за допомогою спеціального посередника – адаптера, наприклад `ArrayAdapter`. Адаптер приймає контекст, шаблон розмітки для одного рядка та сам масив даних, після чого призначається списку через властивість `adapter`, що змушує `ListView` автоматично згенерувати та вивести на екран відповідну кількість рядків.

3. **Як реалізувати власний кастомізований список?**

   Реалізація кастомізованого списку вимагає створення власного класу адаптера, який зазвичай успадковується від базового класу `BaseAdapter` або `ArrayAdapter`. Окрім цього, необхідно створити окремий XML-файл розмітки, який визначатиме унікальний дизайн для одного елемента списку, наприклад, з кількома текстовими полями та зображеннями. У класі власного адаптера ключовим кроком є перевизначення методу `getView`, де за допомогою `LayoutInflater` створений XML-шаблон перетворюється на реальний об'єкт `View`, після чого його внутрішні елементи наповнюються конкретними даними з масиву для поточної позиції перед тим, як повернути готовий рядок на екран.

4. **Що собою являє `Header` і як його реалізувати в списках?**

   `Header` являє собою спеціальний статичний або динамічний елемент інтерфейсу, який закріплюється у самій верхній частині списку і прокручується разом із рештою його вмісту. Він часто використовується для відображення заголовків, логотипів, панелей пошуку або загальної статистики. Для його реалізації потрібно спочатку створити окремий XML-файл з дизайном заголовка, потім у коді активності перетворити його на об'єкт за допомогою `LayoutInflater` і додати до списку викликом методу `addHeaderView`, обов'язково зробивши це до моменту призначення адаптера списку.

5. **Що собою являє `Footer` і як його реалізувати в списках?**

   `Footer` є логічним аналогом заголовка, але розташовується у самому кінці списку під усіма основними елементами. Його зазвичай застосовують для розміщення кнопок завантаження додаткових даних, інформації про авторські права або підсумкових сум. Технічна реалізація повністю ідентична створенню заголовка: розробник створює дизайн у розмітці, інфлює його у View-елемент в оперативній пам'яті та прикріплює до нижньої частини компонента викликом методу `addFooterView` безпосередньо перед тим, як зв'язати список із його адаптером.

6. **Які Ви знаєте механізми зворотного виклику для обробки подій в**

Android-додатках?

Для обробки подій в Android переважно використовуються слухачі подій (Event Listeners), які є інтерфейсами з методами зворотного виклику. Найпоширенішим є `OnClickListener` для обробки звичайних натискань на елементи екрана. Також існують `OnLongClickListener` для тривалих натискань, `OnTouchListener` для відстеження складніших жестів і рухів пальця по екрану, та `OnItemClickListener`, який застосовується спеціально у списках для ідентифікації натискання на конкретний рядок. Крім того, події можна обробляти через перевизначення системних методів самої активності, таких як `onKeyDown` для апаратних кнопок.

7. **Як передати динамічний масив об'єктів з одного Activity в інше за допомогою Intent?**

   Передача масиву об'єктів через `Intent` вимагає попередньої підготовки самих об'єктів. Якщо це масив стандартних типів (наприклад, рядків), можна використати вбудовані методи на кшталт `putStringArrayListExtra`. Якщо ж масив складається з користувацьких об'єктів, їхній клас обов'язково повинен реалізовувати інтерфейс `Serializable` або, що є набагато ефективнішим в Android, інтерфейс `Parcelable`. Після цього масив таких об'єктів пакується в `Intent` за допомогою методу `putParcelableArrayListExtra`, а в приймаючій активності витягується методом `getParcelableArrayListExtra`, що дозволяє безпечно переносити складні структури даних між екранами.

Лістинг.

AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android">

<uses-permission android:name="android.permission.POST_NOTIFICATIONS" />
<uses-permission android:name="android.permission.SCHEDULE_EXACT_ALARM" />
<uses-permission android:name="android.permission.USE_EXACT_ALARM" />

<application
android:allowBackup="true"
android:icon="@mipmap/ic_launcher"
android:label="@string/app_name"
android:roundIcon="@mipmap/ic_launcher_round"
android:supportsRtl="true"
android:theme="@style/Theme.KPIMMOS">
<activity
android:name=".MainActivity"
android:exported="true">
<intent-filter>
<action android:name="android.intent.action.MAIN" />

<category android:name="android.intent.category.LAUNCHER" />
</intent-filter>
</activity>

<activity
android:name=".DetailActivity"
android:exported="false" />

<receiver
android:name=".ReminderReceiver"
android:exported="false" />
</application>
</manifest>
```

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:padding="16dp">

<LinearLayout
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:orientation="vertical">

<EditText
android:id="@+id/etTheme"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:hint="Тема нагадування" />

<EditText
android:id="@+id/etMessage"
android:layout_width="match_parent"
android:layout_height="wrap_content"
```

```
22  android:hint="Текст повідомлення"
23  android:lines="3" />
24
25  <LinearLayout
26  android:layout_width="match_parent"
27  android:layout_height="wrap_content"
28  android:orientation="horizontal"
29  android:layout_marginTop="8dp">
30
31  <Button
32  android:id="@+id/btnDate"
33  android:layout_width="0dp"
34  android:layout_height="wrap_content"
35  android:layout_weight="1"
36  android:text="Обрати дату" />
37
38  <Button
39  android:id="@+id/btnTime"
40  android:layout_width="0dp"
41  android:layout_height="wrap_content"
42  android:layout_weight="1"
43  android:text="Обрати час" />
44  </LinearLayout>
45
46  <TextView
47  android:id="@+id/tvDateTime"
48  android:layout_width="match_parent"
49  android:layout_height="wrap_content"
50  android:text="Час не обрано"
51  android:textAlignment="center"
52  android:layout_marginTop="8dp" />
53
54  <Button
55  android:id="@+id/btnSave"
56  android:layout_width="match_parent"
57  android:layout_height="wrap_content"
58  android:text="Зберегти нагадування"
59  android:layout_marginTop="16dp" />
60
61  <View
62  android:layout_width="match_parent"
63  android:layout_height="1dp"
64  android:background="#CCCCCC"
65  android:layout_marginVertical="16dp" />
66
67  <EditText
68  android:id="@+id/etDeleteId"
69  android:layout_width="match_parent"
70  android:layout_height="wrap_content"
71  android:hint="ID для видалення"
72  android:inputType="number" />
73
74  <Button
75  android:id="@+id/btnDelete"
76  android:layout_width="match_parent"
77  android:layout_height="wrap_content"
78  android:text="Видалити повідомлення" />
79
80  <TextView
81  android:id="@+id/tvAllReminders"
82  android:layout_width="match_parent"
83  android:layout_height="wrap_content"
84  android:layout_marginTop="16dp"
```

```
android:textSize="16sp" />

</LinearLayout>
</ScrollView>
```

MainActivity.kt

```
package ua.kpi.lab6

import android.app.AlarmManager
import android.app.DatePickerDialog
import android.app.PendingIntent
import android.app.TimePickerDialog
import android.content.Context
import android.content.Intent
import android.os.Bundle
import android.widget.Button
import android.widget.EditText
import android.widget.TextView
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity
import java.text.SimpleDateFormat
import java.util.Calendar
import java.util.Locale

class MainActivity : AppCompatActivity() {

    private lateinit var dbHelper: DatabaseHelper
    private lateinit var tvAllReminders: TextView
    private lateinit var tvDateTime: TextView

    private val calendar = Calendar.getInstance()

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        dbHelper = DatabaseHelper(this)

        val etTheme = findViewById<EditText>(R.id.etTheme)
        val etMessage = findViewById<EditText>(R.id.etMessage)
        val etDeleteId = findViewById<EditText>(R.id.etDeleteId)

        tvAllReminders = findViewById(R.id.tvAllReminders)
        tvDateTime = findViewById(R.id.tvDateTime)

        val btnDate = findViewById<Button>(R.id.btnDate)
        val btnTime = findViewById<Button>(R.id.btnTime)
        val btnSave = findViewById<Button>(R.id.btnSave)
        val btnDelete = findViewById<Button>(R.id.btnDelete)

        updateRemindersList()

        btnDate.setOnClickListener {
            val year = calendar.get(Calendar.YEAR)
            val month = calendar.get(Calendar.MONTH)
            val day = calendar.get(Calendar.DAY_OF_MONTH)

            // Show date picker dialog
            DatePickerDialog(this, { _, selectedYear, selectedMonth, selectedDay ->
                calendar.set(Calendar.YEAR, selectedYear)
                calendar.set(Calendar.MONTH, selectedMonth)
```

```
                calendar.set(Calendar.DAY_OF_MONTH, selectedDay)
                updateDateTimeText()
            }, year, month, day).show()
        }

        btnTime.setOnClickListener {
            val hour = calendar.get(Calendar.HOUR_OF_DAY)
            val minute = calendar.get(Calendar.MINUTE)

            // Show time picker dialog
            TimePickerDialog(this, { _, selectedHour, selectedMinute ->
                calendar.set(Calendar.HOUR_OF_DAY, selectedHour)
                calendar.set(Calendar.MINUTE, selectedMinute)
                updateDateTimeText()
            }, hour, minute, true).show()
        }

        btnSave.setOnClickListener {
            val theme = etTheme.text.toString()
            val message = etMessage.text.toString()
            val timeString = tvDateTime.text.toString()

            if (theme.isNotEmpty() && message.isNotEmpty() && timeString != "Час не обрано") {
                // Save to database
                dbHelper.insertReminder(theme, message, timeString)

                // Schedule system alarm
                scheduleNotification(theme, message, calendar.timeInMillis)

                Toast.makeText(this, "Нагадування збережено", Toast.LENGTH_SHORT).show()
                updateRemindersList()
            } else {
                Toast.makeText(this, "Заповніть всі поля та оберіть час",
                ↪  Toast.LENGTH_SHORT).show()
            }
        }

        btnDelete.setOnClickListener {
            val idStr = etDeleteId.text.toString()
            if (idStr.isNotEmpty()) {
                val success = dbHelper.deleteReminder(idStr.toInt())
                if (success) {
                    Toast.makeText(this, "Видалено успішно", Toast.LENGTH_SHORT).show()
                    updateRemindersList()
                } else {
                    Toast.makeText(this, "Помилка видалення", Toast.LENGTH_SHORT).show()
                }
            }
        }
    }

    // Refresh the text view displaying all database entries
    private fun updateRemindersList() {
        tvAllReminders.text = dbHelper.getAllReminders()
    }

    // Update the UI text indicating chosen time
    private fun updateDateTimeText() {
        val format = SimpleDateFormat("dd.MM.yyyy HH:mm", Locale.getDefault())
        tvDateTime.text = format.format(calendar.time)
    }

    // Configure AlarmManager to trigger the receiver at specific time
```

```
    private fun scheduleNotification(theme: String, message: String, timeInMillis: Long) {
        val intent = Intent(this, ReminderReceiver::class.java).apply {
            putExtra("theme", theme)
            putExtra("message", message)
        }

        val pendingIntent = PendingIntent.getBroadcast(
        this,
        timeInMillis.toInt(),
        intent,
        PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE
        )

        val alarmManager = getSystemService(Context.ALARM_SERVICE) as AlarmManager

        // Use exact alarm for precise notification timing
        try {
            alarmManager.setExact(AlarmManager.RTC_WAKEUP, timeInMillis, pendingIntent)
        } catch (e: SecurityException) {
            Toast.makeText(this, "Немає дозволу на точні будильники", Toast.LENGTH_SHORT).show()
        }
    }
}
```

activity_detail.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:orientation="vertical"
android:padding="24dp"
android:gravity="center">

<TextView
android:id="@+id/tvDetailTheme"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:textSize="24sp"
android:textStyle="bold"
android:layout_marginBottom="16dp"/>

<TextView
android:id="@+id/tvDetailMessage"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:textSize="18sp"
android:textAlignment="center"/>

</LinearLayout>
```

DetailActivity.kt

```
package ua.kpi.lab6

import android.os.Bundle
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity

class DetailActivity : AppCompatActivity() {
```

```

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_detail)

        val tvDetailTheme = findViewById<TextView>(R.id.tvDetailTheme)
        val tvDetailMessage = findViewById<TextView>(R.id.tvDetailMessage)

        // Extract data passed from the notification intent
        val theme = intent.getStringExtra("theme") ?: "Без теми"
        val message = intent.getStringExtra("message") ?: "Немає тексту"

        tvDetailTheme.text = theme
        tvDetailMessage.text = message
    }
}
```

DatabaseHelper.kt

```
package ua.kpi.lab6

import android.content.ContentValues
import android.content.Context
import android.database.sqlite.SQLiteDatabase
import android.database.sqlite.SQLiteOpenHelper

// Helper class to manage the SQLite database for reminders
class DatabaseHelper(context: Context) :
SQLiteOpenHelper(context, DATABASE_NAME, null, DATABASE_VERSION) {

    companion object {
        private const val DATABASE_NAME = "reminders.db"
        private const val DATABASE_VERSION = 1

        const val TABLE_NAME = "reminders_table"
        const val COLUMN_ID = "id"
        const val COLUMN_THEME = "theme"
        const val COLUMN_MESSAGE = "message"
        const val COLUMN_DATE = "remind_date"
    }

    override fun onCreate(db: SQLiteDatabase) {
        // Create table query for reminders
        val createTableQuery = ("CREATE TABLE $TABLE_NAME ("
        + "$COLUMN_ID INTEGER PRIMARY KEY AUTOINCREMENT, "
        + "$COLUMN_THEME TEXT, "
        + "$COLUMN_MESSAGE TEXT, "
        + "$COLUMN_DATE TEXT)")
        db.execSQL(createTableQuery)
    }

    override fun onUpgrade(db: SQLiteDatabase, oldVersion: Int, newVersion: Int) {
        db.execSQL("DROP TABLE IF EXISTS $TABLE_NAME")
        onCreate(db)
    }

    // Insert a new reminder into the database
    fun insertReminder(theme: String, message: String, date: String): Long {
        val db = this.writableDatabase
        val values = ContentValues()
        values.put(COLUMN_THEME, theme)
        values.put(COLUMN_MESSAGE, message)
```

```
        values.put(COLUMN_DATE, date)
        val id = db.insert(TABLE_NAME, null, values)
        db.close()
        return id
    }

    // Retrieve all reminders as a formatted string
    fun getAllReminders(): String {
        val db = this.readableDatabase
        val cursor = db.rawQuery("SELECT * FROM $TABLE_NAME ORDER BY $COLUMN_ID DESC", null)
        val stringBuilder = StringBuilder()

        if (cursor.moveToFirst()) {
            do {
                val id = cursor.getInt(cursor.getColumnIndexOrThrow(COLUMN_ID))
                val theme = cursor.getString(cursor.getColumnIndexOrThrow(COLUMN_THEME))
                val message = cursor.getString(cursor.getColumnIndexOrThrow(COLUMN_MESSAGE))
                val date = cursor.getString(cursor.getColumnIndexOrThrow(COLUMN_DATE))
                stringBuilder.append("ID: $id | Тема: $theme\nЧас: $date\nТекст: $message\n\n")
            } while (cursor.moveToNext())
        } else {
            stringBuilder.append("Немає збережених нагадувань.")
        }
        cursor.close()
        db.close()
        return stringBuilder.toString()
    }

    // Delete a specific reminder by its ID
    fun deleteReminder(id: Int): Boolean {
        val db = this.writableDatabase
        val result = db.delete(TABLE_NAME, "$COLUMN_ID = ?", arrayOf(id.toString()))
        db.close()
        return result > 0
    }
}
```

ReminderReceiver.kt

```
package ua.kpi.lab6

import android.app.NotificationChannel
import android.app.NotificationManager
import android.app.PendingIntent
import android.content.BroadcastReceiver
import android.content.Context
import android.content.Intent
import android.os.Build
import androidx.core.app.NotificationCompat

// Receiver triggered by AlarmManager to show the notification
class ReminderReceiver : BroadcastReceiver() {

    override fun onReceive(context: Context, intent: Intent) {
        val theme = intent.getStringExtra("theme") ?: "Нагадування"
        val message = intent.getStringExtra("message") ?: ""

        val notificationManager =
        context.getSystemService(Context.NOTIFICATION_SERVICE) as NotificationManager
        val channelId = "reminder_channel"

        // Create notification channel for modern Android versions
```

```
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
            val channel =
            NotificationChannel(channelId, "Reminders", NotificationManager.IMPORTANCE_HIGH)
            notificationManager.createNotificationChannel(channel)
        }

        // Intent to open DetailActivity when notification is clicked
        val detailIntent = Intent(context, DetailActivity::class.java).apply {
            flags = Intent.FLAG_ACTIVITY_NEW_TASK or Intent.FLAG_ACTIVITY_CLEAR_TASK
            putExtra("theme", theme)
            putExtra("message", message)
        }

        val pendingIntent = PendingIntent.getActivity(
        context,
        System.currentTimeMillis().toInt(),
        detailIntent,
        PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE
        )

        // Build the notification with a custom icon
        val builder = NotificationCompat.Builder(context, channelId)
        .setSmallIcon(R.drawable.ic_notification) // Ensure this icon exists in res/drawable
        .setContentTitle(theme)
        .setContentText(message)
        .setPriority(NotificationCompat.PRIORITY_HIGH)
        .setAutoCancel(true)
        .setContentIntent(pendingIntent)

        notificationManager.notify(System.currentTimeMillis().toInt(), builder.build())
    }
}
```